

[CONCEPT]

What is Web Cache Deception?

THE BIG PICTURE

You trick a cache into **storing someone else's private page** as if it were a public static file. Then you read the cached copy and steal their data (API keys, CSRF tokens, personal info).

Real-world analogy:

Imagine a post office that caches letters addressed to ".pdf" endings without opening them. You label your envelope "mybank.com/account/statement.pdf" — the post office stores a copy. The bank sees only "mybank.com/account" and puts Alice's bank statement inside. You pick up the cached copy and read Alice's statement.

KEY DISTINCTION

Web Cache Deception vs Web Cache Poisoning — easy to confuse:

■ CACHE POISONING

You inject malicious content into the cache. Other users visit a normal URL and get YOUR malicious response.

Think: poisoning the water supply

■ CACHE DECEPTION (THIS TOPIC)

You trick cache into storing the VICTIM's private response. You then read their data from cache.

Think: photocopying someone's mail

CACHE RULES — WHAT GETS STORED

How a cache decides what to store — Cache Rules (what triggers caching):

```
; Rule type 1 – Static file extensions:
Cache stores the response if URL ends in: .js .css .png .ico .exe .svg

; Rule type 2 – Static directory prefix:
Cache stores if URL starts with: /static/ /assets/ /resources/ /images/

; Rule type 3 – Exact filename match:
Cache stores if URL is exactly: robots.txt favicon.ico index.html
```

*Key idea: Cache rules look at the URL to decide "is this static?". Origin server looks at the URL to decide "what page do I serve?". **They can disagree** — that's the bug.*

CACHE KEYS & DETECTION

Cache Key — how cache decides if two requests are "the same":

Usually = URL path + query params. If two requests have the same cache key → second request gets the first's cached response.

Detecting if a response was cached:

- **X-Cache: miss** = just fetched from origin (and probably just got cached)
- **X-Cache: hit** = served directly from cache (someone's response is stored!)
- **Cache-Control: max-age=30** = cacheable for 30 seconds
- **Cache-Control: no-store** = should NOT be cached (correctly secured)

Tip: Always use Param Miner → "Add dynamic cachebuster" while testing. It adds a unique query string per request so you never accidentally read someone else's stale cache.

[LAB 1 — APPRENTICE]

Path Mapping Discrepancy

CONCEPT

The core idea in plain English:

Modern web apps use REST-style routing. A URL like `/my-account/literally-anything` gets routed to the same handler as `/my-account`. The extra bit after the slash is ignored by the origin server. But the cache doesn't know this — it just looks at the file extension.

Real-world analogy:

You tell a receptionist "please hold any letters for room 42/abc.js". The building's system routes all "room 42/ANYTHING" to just room 42. But the receptionist files it under "abc.js" thinking it's a JS file. You come later and ask for "room 42/abc.js" — receptionist hands you the cached copy.

WHAT HAPPENS UNDER THE HOOD

Step-by-step: what happens with the URL `/my-account/app.js`

1. REQUEST ARRIVES

GET `/my-account/app.js`
User visits this URL (logged in as victim)

2. CACHE CHECKS IT

Ends in `.js`?
YES → apply `.js` cache rule → store this response!

3. ORIGIN HANDLES IT

Regex: `/my-account/*`
Matches! Ignores `/app.js` → serves `/my-account` with API key

Result: origin returned the private `/my-account` page, but cache stored it thinking it was a `.js` file. Now YOU request the same URL without being logged in → cache serves you the victim's API key.

LAB SOLUTION

Lab 1 — Walkthrough (Goal: steal carlos's API key)

- 01 Log in as wiener:peter. Notice your API key on /my-account.
- 02 In Burp Repeater, change the path to /my-account/abc and send it. Still get 200 + API key?
→ **Origin abstracts the path!** (this is what makes Lab 1 exploitable)
- 03 Now try /my-account/abc.js → response has **X-Cache: miss** + **Cache-Control: max-age=30** → **it just got cached!**
- 04 Send the request again → **X-Cache: hit** → confirmed: this URL is now being cached ✓
- 05 Deliver this URL to carlos (use exploit server): `https://[lab-url]/my-account/abc.js`
- 06 Wait for carlos to visit. Then YOU request /my-account/abc.js without logging in → cache returns carlos's page → grab his API key

```
; Quick test to know if Lab 1 technique will work:
GET /my-account/abc → 200 + API key? YES → origin abstracts (Lab 1 works!)
GET /my-account/abc → 404? NO → try Lab 2 delimiter technique

; Then confirm the cache stores .js:
GET /my-account/abc.js → X-Cache: miss + Cache-Control: max-age ? → cached!
→ resend → X-Cache: hit? → confirmed ✓
```

[LAB 2 — PRACTITIONER]

Delimiter Discrepancy

CONCEPT

The core idea in plain English:

A "delimiter" is a character that says "the URL path ends here, the rest is something else". Different frameworks use different delimiter characters. If the origin server uses ; as a delimiter but the cache doesn't, we can put a fake .js extension after the semicolon.

Real-world analogy:

A library catalog system uses ";" to separate book title from metadata.

"HarryPotter;illustrated_edition.jpg" → librarian reads title as "HarryPotter". But the filing robot doesn't know about ";" — it sees a .jpg and files it in the photo cabinet. You request that filing number and find Harry Potter's contents in the photo cabinet.

Different frameworks use different delimiter characters:

```
; Java Spring → uses ; (for "matrix variables": /path;param=value)
; Ruby on Rails → uses . (for response format: /profile.json)
; OpenLiteSpeed → uses %00 (encoded null character)
; Standard (all) → uses ? (query string start – cache also knows this one!)
```

LAB 1 VS LAB 2

Why Lab 2 is different from Lab 1:

LAB 1 – PATH ABSTRACTION

GET /my-account/abc → 200 + data

Origin ignores the /abc bit

Can directly append .js after a slash

LAB 2 – NO ABSTRACTION

GET /my-account/abc → 404 Not Found

Origin rejects unknown paths

Need ; to trick origin into ignoring the .js

WHAT HAPPENS UNDER THE HOOD

Step-by-step: what happens with /my-account;abc.js

ORIGIN (JAVA SPRING)

/my-account;abc.js

Sees ; → delimiter!

Truncates to /my-account

Returns API key page ✓

CACHE (NOT SPRING)

/my-account;abc.js

No ; rule → sees full path

Ends in .js → cache rule!

Stores the response ✓

ATTACKER

GET /my-account;abc.js

No session cookie

Cache hit → gets victim

data from cache ✓

LAB SOLUTION

Lab 2 — Walkthrough (Goal: steal carlos's API key)

- 01 Log in as wiener:peter. Confirm API key visible on /my-account.
- 02 Try /my-account/abc → 404 Not Found → origin does NOT abstract → Lab 1 trick won't work here.
- 03 Open Burp Intruder (Sniper attack). Set path to /my-account\$\$abc (payload between my-account and abc). **Disable "URL-encode these characters"** in Payload Encoding — critical or delimiters get mangled!
- 04 Use PortSwigger's "WCD delimiter list" as payload. Run attack. Look for responses returning 200 + API key. These chars are origin-side delimiters.
- 05 Likely results: both ; and ? give 200. Now test which one also gets CACHED.
- 06 Try /my-account;test.js → check for X-Cache header. Try /my-account?test.js → no cache headers.
- 07 **Why ? fails:** ? is a universal delimiter — the cache ALSO strips the query string, so .js never enters the cache key. ; is origin-only so cache sees the full .js extension.
- 08 Exploit URL: /my-account;abc.js. Deliver to carlos, wait, GET unauthenticated → carlos's API key.

; Summary of the ; vs ? difference:

GET /my-account?test.js

; Cache: ? is a universal delimiter → cache key = /my-account (no .js)

; → NOT cached ✗

GET /my-account;test.js

; Cache: doesn't know about ; → sees full /my-account;test.js → ends in .js

; → CACHED ✓ → origin gets /my-account → returns API key page

DELIMITER DECODING VARIANT

Bonus: Delimiter Decoding Discrepancy — when the delimiter char gets URL-encoded

Some servers decode %XX encoded chars before treating them as delimiters. You can use the encoded form to bypass a cache that doesn't decode.

```
; Example: origin uses # as delimiter AND decodes %23
GET /profile%23wcd.css
; Origin: decodes %23 → # → delimiter → serves /profile (private page)
; Cache: sees %23 as literal text → path = /profile%23wcd.css → .css rule →
  CACHED

; Example: cache decodes THEN applies rule, then forwards decoded URL to origin
GET /myaccount%3fwcd.css ; %3f = encoded ?
; Cache: applies .css rule to /myaccount%3fwcd.css → stores it
; then decodes and forwards /myaccount?wcd.css to origin
; Origin: sees ? → delimiter → serves /myaccount (private page)
```

[LAB 3 — PRACTITIONER]

Exploiting Origin Server Normalization

CONCEPT

The core idea in plain English:

Normalization = cleaning up a URL path. This includes decoding %XX encoded characters and resolving dot-segments (. . / = "go up one directory"). If the origin normalizes but the cache doesn't, we can craft a path traversal that the origin resolves to a private endpoint, but the cache stores under a static directory rule.

Real-world analogy:

A smart receptionist (origin) interprets "floor3/storage/./executive" as "floor3/executive" (goes up one level). A dumb filing robot (cache) just sees the folder starts with "/storage" so it files it in the storage cabinet. You request that cabinet slot and get the executive's confidential files.

Why use %2f instead of / in the payload?

If you write . . / directly in the URL, your browser resolves it before the request even leaves. So you write . . %2f — %2f is an encoded slash. The origin decodes it and resolves the traversal. The cache (which doesn't decode) just sees a weird string.

WHAT HAPPENS UNDER THE HOOD

Step-by-step: what happens with /resources/..%2fmy-account

CACHE SEES

```
/resources/..%2fmy-account
Starts with /resources/
→ /resources cache rule!
→ STORES the response
Doesn't decode %2f or resolve ..
```

ORIGIN SEES

```
/resources/..%2fmy-account
Decodes %2f → /
Resolves ../ → goes up
Resulting path: /my-account
→ Returns private API key page
Decodes and resolves everything
```

Lab 3 — Walkthrough

- 01 Confirm no path abstraction: `/my-account/abc` → 404
- 02 Run delimiter Intruder → only `?` returns 200. Since `?` is a universal delimiter, cache also uses it → no useful discrepancy.
- 03 Test if origin normalizes — use an UNENCODED traversal (will be resolved by browser too, but we just want to know): `GET /aaa/./my-account` → 200 + API key? → origin normalizes ✓
- 04 Find the static directory: in HTTP History, filter for cached responses + script/image/CSS MIME types. Notice many requests start with `/resources` and have cache headers.
- 05 Verify cache does NOT normalize: send `/aaa/..%2fresources/js/app.js` twice. First is miss, second should be **miss again** (not hit) → cache didn't resolve the path → it doesn't normalize ✓
- 06 Craft exploit URL: `/resources/..%2fmy-account`
- 07 Deliver to carlos → wait → GET unauthenticated → cached API key

```
; Why ..%2f and not ../?
; ../ → your browser resolves this → becomes just /my-account → no traversal
trick possible
; ..%2f → browser doesn't resolve encoded slashes → reaches cache as-is
; cache: stores under /resources rule
; origin: decodes %2f → / → resolves ../ → gets to /my-account ✓
; Exploit:
GET /resources/..%2fmy-account
; Cache interprets as: /resources/..%2fmy-account → /resources prefix → STORE
; Origin interprets as: /resources/./my-account → /my-account → return page
```

[LAB 4 — PRACTITIONER]

Exploiting Cache Server Normalization

CONCEPT

The core idea in plain English:

Now it's the **opposite** — the CACHE normalizes (resolves dot-segments) but the ORIGIN doesn't. If we use a path traversal alone, the cache resolves it to /resources (good for caching) but the origin sees the raw encoded path and returns an error. So we ALSO need a delimiter that the origin uses (to strip the junk) but the cache ignores.

Real-world analogy:

Smart filing robot (cache) reads "/account;navigate-up/documents" and understands the path traversal → files under /documents. Dumb receptionist (origin) sees the semicolon as a separator → reads only "/account" → hands you account data. The robot stored account data in the documents cabinet. You pull it out.

Why path traversal alone is NOT enough here:

```
; BROKEN attempt (no delimiter):
GET /my-account%2f%2e%2e%2fresources
; Cache: decodes + resolves → /resources → stores ✓
; Origin: does NOT decode → sees /my-account%2f%2e%2e%2fresources → ERROR X
; What we need – add a delimiter the origin understands:
GET /my-account;%2f%2e%2e%2fresources
; Cache: no ; rule → sees full path → decodes + resolves → /resources → STORES ✓
; Origin: sees ; → delimiter → truncates to /my-account → returns API key ✓
```

ENCODING EXPLAINED

Why encode ALL of %2f%2e%2e%2f and not just the slash?

If you write / . . / (unencoded dots and slash), the browser resolves it before the request leaves. You must encode everything: %2f=/ and %2e%2e=.. so it passes through the browser untouched. The cache then decodes and resolves it server-side.

```
; Encoding reference:
%2f = / (forward slash)
%2e = . (dot)
%2e%2e%2f = ../ (dot-dot-slash = go up one directory)
; So the full traversal sequence ../ encoded = %2e%2e%2f
; Or sometimes: ../%2f (only the slash encoded – works when browser won't resolve it)
```

LAB SOLUTION

Lab 4 — Walkthrough

- 01 Confirm no path abstraction: /my-account/abc → 404
- 02 Run delimiter Intruder → ; returns 200 + API key
- 03 Confirm origin does NOT normalize: GET /aaa/..%2fmy-account → 404 (origin didn't decode/resolve it)
- 04 Find static dir: /resources in HTTP History
- 05 Check if cache normalizes: send /aaa/..%2fresources/js/app.js twice. Second request → X-Cache: hit? → cache resolved the traversal → cache normalizes ✓
- 06 Build exploit: /my-account;%2f%2e%2e%2fresources
- 07 Deliver to carlos → GET unauthenticated → cached API key

```
; Full exploit breakdown:
GET /my-account;%2f%2e%2e%2fresources

; Cache parses:
; No ; delimiter → considers full path: /my-account;%2f%2e%2e%2fresources
; Decodes: /my-account;../resources
; Resolves ../: /resources
; Matches /resources cache rule → STORES ✓

; Origin parses:
; Sees ; → delimiter → truncates to /my-account
; Returns private API key page ✓

; Net result: victim's /my-account page is cached under the /resources rule
```

[DEFENSE & REFERENCE]

Quick Reference

Decision tree — run these 4 tests on any target

SUMMARY TABLE

All 4 labs at a glance

MITIGATIONS

Prevention

1. Set Cache-Control: no-store, private on all authenticated / dynamic endpoints
2. CDN config: don't let CDN rules override Cache-Control headers from origin
3. Enable CDN built-in protection — e.g. **Cloudflare Cache Deception Armor** verifies Content-Type matches URL extension
4. Ensure cache and origin parse URLs identically — same delimiter chars, same normalization behavior